EXPERIMENT-69(HDOC)

C Programming

Program: Digital Root of a Number Using Recursion and a Static Call Counter

Step 1: Understand the Problem Statement

- Input: A non-negative integer n. A negative number or a non-numeric value is rejected with an error message.
- Output: Every reduction step (for example `9875 => 29`), the digital root of the number, and the number of recursive calls made.
- The digital root is found by replacing the number with the sum of its digits again and again until only one digit is left. For 9875: 9 + 8 + 7 + 5 = 29, then 2 + 9 = 11, then 1 + 1 = 2, so the digital root is 2.
- Two recursive functions are used and no loop appears anywhere: `sumOfDigits(n)` returns `n % 10 + sumOfDigits(n / 10)` and stops at 0, and `digitalRoot(n)` calls itself with the digit sum until the number has a single digit.
- The recursive calls are counted with a static variable, `static int callCount`. A static variable is created once, is private to the file and keeps its value for the whole run, so every call of `digitalRoot()` updates the same counter. It is increased each time `digitalRoot()` calls itself; the first call from main() is not a recursive call and is not counted.

Rule(s) used: digitalRoot(n) = n when n < 10; otherwise digitalRoot(sumOfDigits(n)). sumOfDigits(0) = 0 and sumOfDigits(n) = n % 10 + sumOfDigits(n / 10). Number of recursive calls = number of reduction steps (the 9875 example needs 3: 9875 → 29 → 11 → 2).

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Number: 9875`	9875 => 29 29 => 11 11 => 2 Digital root of 9875 = 2 Number of recursive calls = 3	9875 => 29 29 => 11 11 => 2 Digital root of 9875 = 2 Number of recursive calls = 3	Pass
2	`Number: 0`	Digital root of 0 = 0 Number of recursive calls = 0	Digital root of 0 = 0 Number of recursive calls = 0	Pass
3	`Number: 7`	Digital root of 7 = 7 Number of recursive calls = 0	Digital root of 7 = 7 Number of recursive calls = 0	Pass
4	`Number: 29`	29 => 11 11 => 2 Digital root of 29 = 2 Number of recursive calls = 2	29 => 11 11 => 2 Digital root of 29 = 2 Number of recursive calls = 2	Pass

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
5	Number: 123456789	123456789 => 45 45 => 9 Digital root of 123456789 = 9 Number of recursive calls = 2	123456789 => 45 45 => 9 Digital root of 123456789 = 9 Number of recursive calls = 2	Pass
6	Number: 2147483647	2147483647 => 46 46 => 10 10 => 1 Digital root of 2147483647 = 1 Number of recursive calls = 3	2147483647 => 46 46 => 10 10 => 1 Digital root of 2147483647 = 1 Number of recursive calls = 3	Pass
7	Number: -5	Invalid input. Enter a non-negative integer.	Invalid input. Enter a non-negative integer.	Pass

Step 3: Algorithm

1. Start the program.
2. Read the number n; if it is not a non-negative integer, display an error message and stop.
3. Set the static counter `callCount = 0` and call `digitalRoot(n)`.
4. In `digitalRoot()`: if `n < 10`, return n (base case: a single digit).
5. Otherwise compute `sum = sumOfDigits(n)`, where `sumOfDigits()` returns 0 for n = 0 and `n % 10 + sumOfDigits(n / 10)` otherwise.
6. Display the step `n => sum`, increase `callCount` by 1 and return `digitalRoot(sum)` (recursive call).
7. When the recursion returns, display `Digital root of n = result` and `Number of recursive calls = callCount`.
8. Stop the program.

Evaluation against Step 2 test cases: for 9875 the digit sum is 29, then 11, then 2, so the digital root is 2 after 3 recursive calls. For 0 and 7 the base case applies at once, so the answer is the number itself with 0 calls. For 29 the steps are 29 => 11 and 11 => 2, giving 2 with 2 calls. For 123456789 the digit sum is 45 and then 9 (2 calls), and for 2147483647 the sums are 46, 10 and 1, giving 1 with 3 calls. A negative number is rejected as invalid input — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

```
/* HDOC Day-70: Digital root of a number using RECURSION, with a STATIC
   variable that counts the number of recursive calls.

   Digital root: repeatedly replace the number by the sum of its digits until
   a single digit is left.   Example:  9875 => 29 => 11 => 2

       digitalRoot(n):  if n < 10        -> n is the answer (base case)
```

```
                        otherwise        -> digitalRoot(sum of the digits of n)
      sumOfDigits(n):   if n == 0        -> 0
                        otherwise        -> n % 10 + sumOfDigits(n / 10)

   No loop is used: both steps are recursive. callCount is a static variable
   (private to this file, it keeps its value between function calls) and is
   increased every time digitalRoot() calls itself.
*/

#include <stdio.h>

/* static: visible only in this file, keeps its value for the whole run */
static int callCount = 0;

/* Function prototypes */
int sumOfDigits(int n);
int digitalRoot(int n);

int main() {
    int n;

    printf("Enter a non-negative integer: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Invalid input. Enter a non-negative integer.\n");
        return 1;
    }

    callCount = 0;                    /* start counting from zero */
    printf("Digital root of %d = %d\n", n, digitalRoot(n));
    printf("Number of recursive calls = %d\n", callCount);

    return 0;
}

/* Returns the sum of the digits of n recursively */
int sumOfDigits(int n) {
    if (n == 0)                       /* base case: no digits left */
        return 0;
    return n % 10 + sumOfDigits(n / 10);
}

/* Returns the digital root of n; counts its recursive calls in callCount */
int digitalRoot(int n) {
    int sum;

    if (n < 10)                       /* base case: a single digit */
        return n;

    sum = sumOfDigits(n);
    printf("%d => %d\n", n, sum);     /* show each reduction step */

    callCount++;                      /* one more recursive call follows */
    return digitalRoot(sum);
}
```

Step 5: Compile, Execute and Evaluate

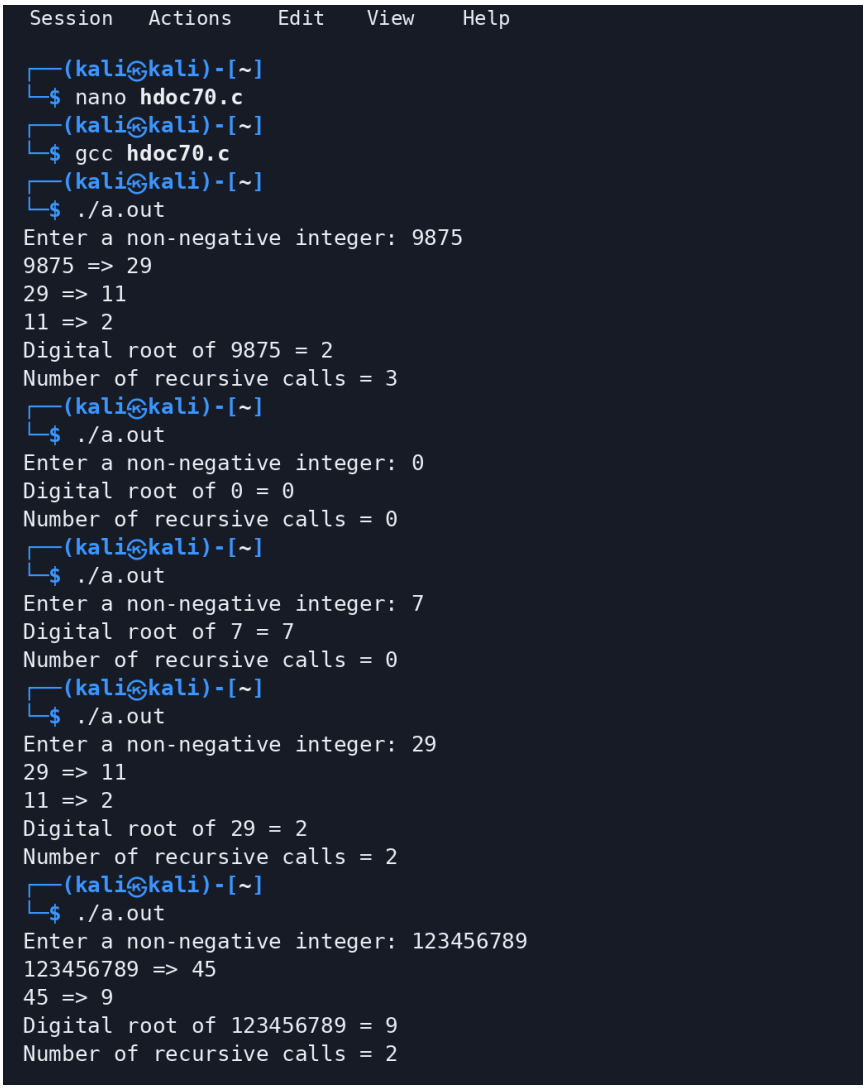
The program was compiled using the GCC/cc compiler and executed from the Linux terminal using the following commands:

cc hdoc70.c	(compile)
./a.out	(execute)

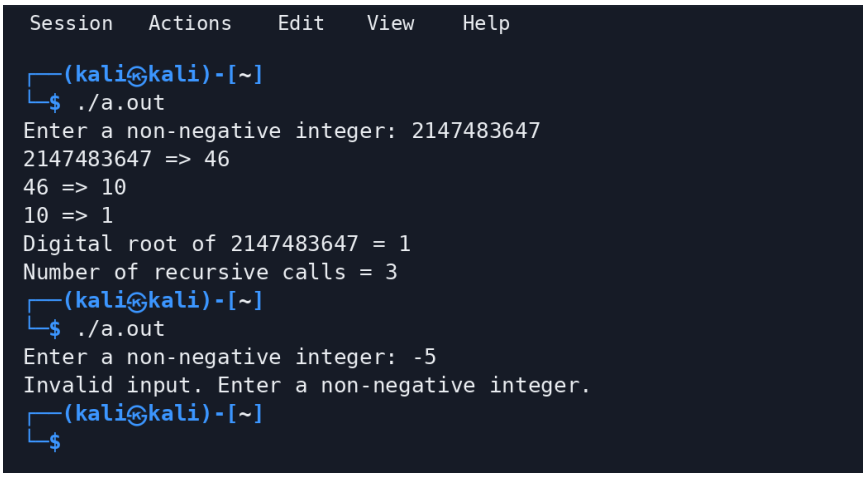
All 7 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 2



Screenshot 2 of 2



Conclusion

In this experiment, a recursive function was written to find the digital root of a number by repeatedly replacing it with the sum of its digits until a single digit remained, using a second recursive function for the digit sum and a static variable to count the recursive calls. The program was compiled with gcc and tested on seven inputs, including the 9875 example from the question (digital root 2 after 3 recursive calls), and every output matched the expected result.